Лабораторная работа №1

Адаптивное численное интегрирование

Компонент EcoIntegralSimpson, задание 35

Содержание

1. Постановка задачи 2
2. Формула Симпсона и оценка погрешности 3
3. Адаптивное уточнение и сложность 4
4. Публичный интерфейс 5
5. Реализация компонента ACOM 6
6. Тестирование и независимый эталон 7
7. Результаты вычислений 8
8. Выводы и воспроизведение 11
9. Использование ИИ в работе 11

Выполнил: Михаил Сибиряков

Группа: 24ТИДИ

2026

1. Постановка задачи

Задание 35 лабораторной работы №1 предусматривает численное адаптивное интегрирование: реализацию метода Гаусса — Кронрода или метода Симпсона. В работе выбран метод Симпсона. Разработан один компонент `EcoIntegralSimpson` с новым публичным интерфейсом `IEcoAdvancedMath` в рамках компонентной модели ACOM.

Необходимо вычислить определённый интеграл

$$I = \int_a^b f(x)\,dx$$

с заданными абсолютным и относительным допусками. Сетка должна уточняться в процессе вычисления в зависимости от поведения функции. Поэтому применение формулы Симпсона только на заранее заданной равномерной сетке не решает поставленную задачу.

Требования к реализации

Компонент и консольный тестовый клиент написаны на C89. Они используют публичные интерфейсы Eco для получения компонентов, выделения памяти и вывода результатов; функции стандартной библиотеки C в собственном коде не вызываются. Компонент собран как статическая библиотека, подключённая к консольному UnitTest.

Для различных числовых типов предусмотрены три метода: `integral` для `double_t`, `integralf` для `float_t` и `integrall` для `long double`. Это числовые варианты одного алгоритма, реализованные в одном компоненте. Методы возвращают статус, а значение интеграла и оценку абсолютной погрешности записывают в выходные параметры.

Область применения

Поддерживаются непрерывные вещественные функции с конечными значениями на конечном отрезке, включая его концы. Несобственные интегралы и специальные преобразования для разрывов или особенностей не реализованы. Непрерывность функции остаётся условием использования: конечное число вычислений не позволяет проверить её на всём отрезке.

При корректных входных данных и $a = b$ возвращаются интеграл 0 и оценка погрешности 0 без вызова функции. При $a > b$ вычисление выполняется на $[b, a]$, а знак результата меняется; оценка абсолютной погрешности остаётся неотрицательной.

Для независимой проверки и сравнения выбрана Python-библиотека mpmath, которая позволяла оценить результаты собственной реализации.

2. Формула Симпсона и оценка погрешности

На одном отрезке $[u, v]$ формула Симпсона имеет вид

$$S(u,v) = \frac{v-u}{6}\left(f(u) + 4f\left(\frac{u+v}{2}\right) + f(v)\right).$$

Она точно интегрирует многочлены степени не выше трёх в точной арифметике.

Пусть $m = (u+v)/2$. Для оценки ошибки сравниваются два приближения:

$$Q_1 = S(u,v), \qquad Q_2 = S(u,m) + S(m,v).$$

Для локального результата используется Q_2, то есть сумма формул на двух половинах. Поправка Ричардсона к значению интеграла не добавляется. В такой проверке участвуют пять узлов: границы, середина и середины половин.

Почему разность делится на 15

Для достаточно гладкой функции ведущая ошибка составной формулы Симпсона пропорциональна четвёртой степени шага. При уменьшении шага вдвое она сокращается приблизительно в 16 раз. Если обозначить ошибку уточнённого приближения через e, то

$$Q_1 \approx I_{[u,v]} + 16e, \qquad Q_2 \approx I_{[u,v]} + e.$$

Вычитание устраняет неизвестный интеграл и даёт $Q_1 - Q_2 \approx 15e$. Поэтому доступная оценка ошибки Q_2 равна

$$E_{\text{quad}} = \frac{|Q_2 - Q_1|}{15}.$$

Истинное значение интеграла для этого вычисления не требуется. Это модель ошибки, а не её точное измерение; для одной лишь непрерывности она не даёт универсальной гарантии.

Учёт округления

Совпадение Q_1 и Q_2 в машинной арифметике не означает нулевую ошибку. Поэтому локальная оценка имеет нижний порог:

$$E_{\text{loc}} = \max\left(\frac{|Q_2 - Q_1|}{15},\, 4\varepsilon_{\text{mach}} A\right),$$

где A вычисляется той же пятиузловой квадратурой для $|f|$. Порог зависит от масштаба функции и точности числового типа. Он также является эвристикой; фактическая ошибка измеряется отдельно при сравнении с независимым эталоном.

3. Адаптивное уточнение и сложность

На текущем разбиении отрезка хранятся локальные значения Q_i и оценки E_i. Приближение всего интеграла и глобальная оценка определяются суммами

$$Q = \sum_i Q_i, \qquad E = \sum_i E_i.$$

Вычисление завершается при выполнении критерия

$$E \le \max\bigl(\varepsilon_{\text{abs}}, \varepsilon_{\text{rel}}|Q|\bigr).$$

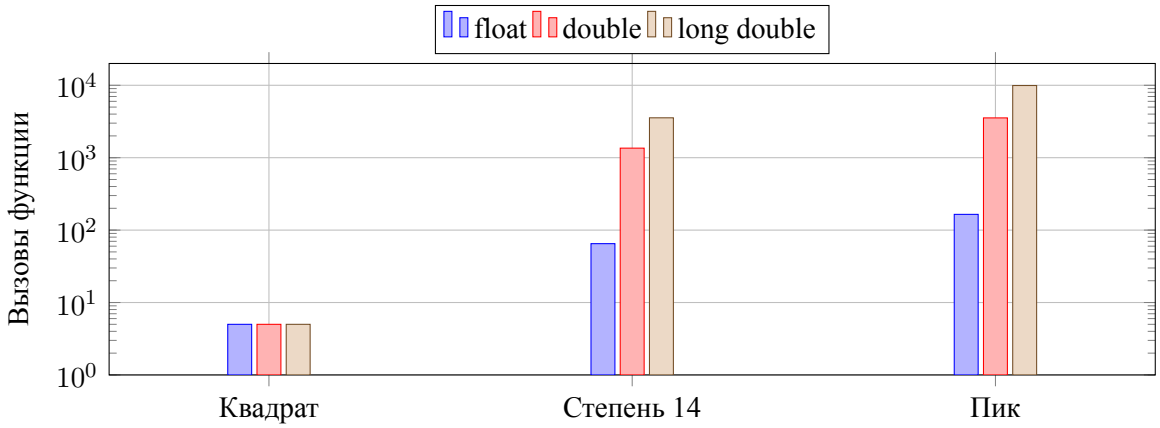
Абсолютный допуск задаётся в единицах интеграла, относительный — как безразмерная доля. При интеграле около нуля абсолютный допуск позволяет сохранить осмысленную цель точности.

Алгоритм начинает с одного отрезка и уточняет разбиение в следующем цикле:

1. Вычислить Q_2 и оценку ошибки на исходном отрезке.
2. Проверить глобальный критерий; при его выполнении вернуть Q и E.
3. Иначе найти подотрезок с наибольшим E_i и разделить его пополам.
4. Вычислить приближения и оценки для обеих половин, заменить ими исходный подотрезок и повторить шаг 2.

Подотрезки хранятся в массиве с лимитом 4096. Исчерпание лимита, нечисловое значение функции или невозможность дальнейшего деления из-за округления приводят к ошибке без записи числовых выходов.

Для n подотрезков выполняется $5 + 10(n-1)$ вызовов функции: пять на исходном отрезке и десять при каждом делении. При линейном поиске максимальной ошибки и пересчёте сумм служебная работа занимает $O(n^2)$. Массив занимает $O(M)$ памяти, где $M = 4096$ — заданная вместимость; используются $n \le M$ записей.



Число вызовов функции в тестах; шкала по вертикали логарифмическая. Для типов заданы разные допуски, приведённые в разделе 7.

4. Публичный интерфейс

В начале `IEcoAdvancedMath` расположены три базовых метода: `QueryInterface`, `AddRef` и `Release`. Далее расположены три числовых метода интегрирования. Фрагмент объявления варианта для `double_t`:

```
typedef double_t (ECOCDECLMETHOD *EcoIntegrandD)(
    double_t x, voidptr_t context);

int16_t (ECOCALLMETHOD *integral)(
    IEcoAdvancedMathPtr_t me,
    EcoIntegrandD function,
    voidptr_t context,
    double_t a, double_t b,
    double_t epsabs, double_t epsrel,
    double_t* result, double_t* error);
```

Метод	Тип аргументов и выходов	Callback
`integral`	`double_t`	`EcoIntegrandD`
`integralf`	`float_t`	`EcoIntegrandF`
`integrall`	`long double`	`EcoIntegrandL`

Callback возвращает значение $f(x)$ напрямую. Контекст `voidptr_t` необязателен и передаётся без изменений. Например, для x^2 контекст не нужен, а для cx^2 он может содержать коэффициент c. Функция и контекст заимствованы на время синхронного вызова: компонент их не сохраняет и не освобождает.

Интегратор возвращает `ERR_ECO_SUCCESS` (0) при успехе и ненулевой код при ошибке. Оба выходных числа записываются только при успехе; при отказе сохраняются прежние значения. Указатели на компонент, функцию и оба выхода обязательны. Раздельные переменные для результата и оценки обеспечивает вызывающая сторона.

Допуски конечны и неотрицательны; хотя бы один строго положителен. Нулевой `epsrel` оставляет абсолютный критерий, нулевой `epsabs` — относительный. Все входы проверяются также при равных границах.

В Linux-ветке установленного SDK есть `float_t` и `double_t`, но нет псевдонима `ldouble_t`. Поэтому `long double` используется явно без собственного `typedef` и изменения SDK.

5. Реализация компонента ACOM

Проект `EcoIntegralSimpson` создан расширением Eco Wizard. Пути в отчёте указаны относительно корня этого проекта. Реализация размещена в его скелете; сохранены каталоги мастера, фабрика и идентификатор компонента. Основные файлы распределены следующим образом:

Каталог	Содержимое
`SharedFiles`	Публичные `IEcoAdvancedMath.h`, `EcoAdvancedMath.idl`, идентификатор компонента.
`HeaderFiles`	Внутренние объявления класса и фабрики.
`SourceFiles`	`CEcoIntegralSimpson.c`: числовые правила, адаптивные циклы и управление объектом; отдельный файл фабрики.
`UnitTestFiles`	Консольный клиент и внешние скрипты проверки.
`AssemblyFiles`	Makefile, MakefileExe и оболочка Linux-сборки.
`BuildFiles`	Библиотека, исполняемый клиент и результаты проверок.

CID компонента:

`BA973067-BA95-4E8F-A50B-DDE55BC4E3DF`

IID интерфейса:

`667182A3-5F5C-4F90-9244-C763139ED855`

CID и суффикс внутренних имён `5BC4E3DF` получены от мастера. IID принадлежит новому интерфейсу. Клиент регистрирует статическую фабрику и получает интерфейс через шину:

```
status = bus->pVTbl->RegisterComponent(
    bus, &CID_EcoIntegralSimpson,
    (IEcoUnknown*)GetIEcoComponentFactoryPtr_BA973067BA954E8FA50BDDE55BC4E3DF);
status = bus->pVTbl->QueryComponent(
    bus, &CID_EcoIntegralSimpson, 0,
    &IID_IEcoAdvancedMath, (voidptr_t*)&component);
```

Класс хранит таблицу виртуальных методов, счётчик ссылок и ссылки на необходимые сервисы. Массив подотрезков выделяется через `IEcoMemoryAllocator1` и освобождается при завершении вызова. На время callback удерживается дополнительная ссылка на объект. Клиент использует `IEcoFileManager1`/`IEcoFile1` для вывода.

Реализации трёх типов записаны явными функциями в обычном C-файле. В очистке ресурсов применён `goto Release`, как в шаблонах Eco.

6. Тестирование и независимый эталон

Консольный клиент находится в `UnitTestFiles/SourceFiles`. Его исходный файл — `EcoIntegralSimpson.c`. Он регистрирует фабрику, получает интерфейс и проверяет поведение компонента через публичные методы. Результат запуска:

```
SUMMARY PASS passed=135 failed=0
```

Проверены:

- регистрация, `QueryInterface`, идентичность интерфейсов и независимые экземпляры;
- три числовых типа, аналитические интегралы, знак, масштабы и адаптивность на гладком пике;
- равные и обратные границы, необязательный контекст, абсолютный и относительный критерии;
- обязательные NULL, неверные допуски, NaN и бесконечности, сохранение выходов при отказе;
- недостижимая точность и интервал, который нельзя разделить в выбранном числовом типе.

При тесте недостижимого допуска все три варианта достигают лимита 4096 подотрезков и возвращают ошибку после 40955 вызовов функции.

Для подсчёта вызовов подынтегральной функции в контексте callback тестового клиента хранится счётчик. Перед каждым вычислением интеграла он обнуляется, а при каждом вызове функции увеличивается на единицу. Итоговое значение записывается в журнал теста и переносится в столбец `calls` CSV-таблицы.

Независимое сравнение

Внешний скрипт `compare_reference.py` использует mpmath 1.3.0, метод tanh-sinh и рабочую точность 50 десятичных знаков. Эталон повторно вычисляется с 70 знаками для проверки устойчивости; он также сопоставляется с аналитическими первообразными с порогом расхождения 10^{-45}. Эта библиотека не включена в компонент или C-клиент.

Для каждого типа сравниваются десять примеров. Фактическая ошибка $\delta = |Q - I_{\text{ref}}|$ проверяется по условию

$$\delta \le 0.5 \cdot 10^{-N}, \qquad N = 4,\ 8,\ 10$$

для `float_t`, `double_t`, `long double`. Это согласованный критерий абсолютной точности на выбранных примерах. Десятичные строки разбираются без преобразования в Python `float`, чтобы не потерять дополнительные разряды `long double`.

Итог: $30/30$ сравнений пройдены. Три вспомогательных теста скрипта сравнения также прошли.

7. Результаты вычислений

Все примеры вычислены на $[0, 1]$. В таблицах Q — результат компонента, E — его оценка абсолютной погрешности, δ — ошибка относительно mpmath. Столбец «Вызовов» показывает затраты на вычисление функции. Номера строк соответствуют следующему набору:

№	$f(x)$	Точный интеграл	Допуски
1	1	1	оба
2	x^2	$1/3$	оба
3	x^{14}	$1/15$	оба
4	$\left(1 + 400(x - 3/8)^2\right)^{-1}$	$(\arctan(25/2) + \arctan(15/2))/20$	оба
5	$x - 1/2$	0	оба
6	$-x^2$	$-1/3$	оба
7	$10^{-4}x^2$	$10^{-4}/3$	оба
8	$100x^2$	$100/3$	оба
9	x^2	$1/3$	только абсолютный
10	x^2	$1/3$	только относительный

Интеграл пика равен приблизительно 0.146460556779044095065. В таблицах исходные числа из CSV округлены для чтения.

7.1. Тип float_t

Основные допуски: $\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 2 \cdot 10^{-6}$. В строках 9 и 10 один из них равен нулю.

№	Q	E	δ	Вызовов
1	1	$4.768 \cdot 10^{-7}$	0	5
2	0.333333343	$1.589 \cdot 10^{-7}$	$9.934 \cdot 10^{-9}$	5
3	0.0666685253	$1.281 \cdot 10^{-6}$	$1.859 \cdot 10^{-6}$	65
4	0.146462917	$1.902 \cdot 10^{-6}$	$2.361 \cdot 10^{-6}$	165
5	0	$1.192 \cdot 10^{-7}$	$4.623 \cdot 10^{-60}$	5
6	−0.333333343	$1.589 \cdot 10^{-7}$	$9.934 \cdot 10^{-9}$	5
7	0.0000333333337	$1.589 \cdot 10^{-11}$	$3.706 \cdot 10^{-13}$	5
8	33.3333359	$1.589 \cdot 10^{-5}$	$2.543 \cdot 10^{-6}$	5
9	0.333333343	$1.589 \cdot 10^{-7}$	$9.934 \cdot 10^{-9}$	5
10	0.333333343	$1.589 \cdot 10^{-7}$	$9.934 \cdot 10^{-9}$	5

Все десять результатов прошли цель $\delta \le 5 \cdot 10^{-5}$ (четыре знака после запятой). Максимальная ошибка — $2.5431315 \cdot 10^{-6}$. Для пика и x^{14} понадобилось уточнение. У некоторых результатов $\delta > E$: оценка не является гарантированной верхней границей ошибки.

7.2. Тип double_t

Основные допуски: $\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 10^{-11}$. Цель внешнего сравнения — восемь знаков после запятой, то есть $\delta \le 5 \cdot 10^{-9}$.

№	Q	E	δ	Вызовов
1	1	$8.882 \cdot 10^{-16}$	0	5
2	0.3333333333333333	$2.961 \cdot 10^{-16}$	$1.850 \cdot 10^{-17}$	5
3	0.06666666667666395	$9.962 \cdot 10^{-12}$	$9.997 \cdot 10^{-12}$	1355
4	0.1464605567839573	$9.980 \cdot 10^{-12}$	$4.913 \cdot 10^{-12}$	3545
5	0	$2.220 \cdot 10^{-16}$	$4.623 \cdot 10^{-60}$	5
6	−0.3333333333333333	$2.961 \cdot 10^{-16}$	$1.850 \cdot 10^{-17}$	5
7	0.00003333333333333333	$2.961 \cdot 10^{-20}$	$1.597 \cdot 10^{-21}$	5
8	33.33333333333333	$2.961 \cdot 10^{-14}$	$4.737 \cdot 10^{-15}$	5
9	0.3333333333333333	$2.961 \cdot 10^{-16}$	$1.850 \cdot 10^{-17}$	5
10	0.3333333333333333	$2.961 \cdot 10^{-16}$	$1.850 \cdot 10^{-17}$	5

Все десять результатов прошли проверку. Максимальная фактическая ошибка равна $9.9972799 \cdot 10^{-12}$. На x^{14} выполнено 1355 вызовов функции, на пике — 3545. Для квадратичной функции хватает первоначальных пяти узлов, поскольку формула Симпсона точна для неё в точной арифметике. Остающееся расхождение порядка 10^{-17} связано с представлением чисел и округлением операций.

Например, для x^{14} оценка составляет приблизительно $9.9624 \cdot 10^{-12}$, а фактическая ошибка — $9.9973 \cdot 10^{-12}$. Эти близкие, но различные числа подтверждают необходимость разделять алгоритмическую оценку и результат независимого сравнения.

Для нечётной относительно середины функции $x - 1/2$ компонент возвращает ноль. В mpmath получается остаток порядка 10^{-60} при истинном интеграле 0. Поэтому соответствующая ненулевая δ отражает округление эталонного вычисления, а не существенную ошибку компонента.

Отрицательная функция даёт отрицательный интеграл и неотрицательную оценку ошибки. Примеры с масштабами 10^{-4} и 100 проверяют поведение на малых и больших значениях. Строки 9 и 10 подтверждают работу каждого критерия по отдельности.

Указанные десятичные разряды результата в таблице служат для представления числа. Сравнение выполнено по исходной строке из консольного клиента; округление таблицы не используется при вычислении δ.

7.3. Тип long double

Основные допуски: $\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 2 \cdot 10^{-13}$. Цель — десять знаков после запятой, то есть $\delta \le 5 \cdot 10^{-11}$.

№	Q	E	δ	Вызовов
1	1	$4.337 \cdot 10^{-19}$	0	5
2	0.3333333333333333	$1.446 \cdot 10^{-19}$	$1.437 \cdot 10^{-20}$	5
3	0.06666666666686505023	$1.977 \cdot 10^{-13}$	$1.984 \cdot 10^{-13}$	3555
4	0.146460556779124776	$1.993 \cdot 10^{-13}$	$8.068 \cdot 10^{-14}$	9925
5	0	$1.084 \cdot 10^{-19}$	$4.623 \cdot 10^{-60}$	5
6	−0.3333333333333333	$1.446 \cdot 10^{-19}$	$1.437 \cdot 10^{-20}$	5
7	0.00003333333333333333493	$1.446 \cdot 10^{-23}$	$1.597 \cdot 10^{-21}$	5
8	33.33333333333333333	$1.446 \cdot 10^{-17}$	$1.437 \cdot 10^{-18}$	5
9	0.3333333333333333333	$1.446 \cdot 10^{-19}$	$1.437 \cdot 10^{-20}$	5
10	0.3333333333333333333	$1.446 \cdot 10^{-19}$	$1.437 \cdot 10^{-20}$	5

Все десять результатов прошли сравнение. Максимальная фактическая ошибка равна $1.9838356 \cdot 10^{-13}$. Значения квадратичных интегралов отличаются от эталона меньше, чем в варианте `double_t`, что согласуется с более длинной мантиссой в данном окружении.

Для пика понадобилось 9925 вызовов, для x^{14} — 3555. В этом наборе тестов для `long double` задан более строгий допуск. Поэтому рост числа вызовов не является сравнением скорости типов при одинаковой требуемой точности.

Сводный результат

Тип	Знаков N	Пройдено	Максимальная δ
`float_t`	4	10/10	$2.5431315 \cdot 10^{-6}$
`double_t`	8	10/10	$9.9972799 \cdot 10^{-12}$
`long double`	10	10/10	$1.9838356 \cdot 10^{-13}$

Успех на данном наборе подтверждает выбранные цели проверки, но не гарантирует точность для любой непрерывной функции. Очень узкий пик между узлами может быть пропущен обеими квадратурами. Практические ограничения также задают машинная точность, представимость промежуточных чисел и лимит подотрезков.

8. Выводы и воспроизведение

Реализован компонент адаптивного интегрирования методом Симпсона с публичными методами `integral`, `integralf`, `integrall`. Адаптивность обеспечивается делением подотрезка с наибольшей оценкой ошибки и проверкой общего критерия точности. Интерфейс разделяет статус вычисления, числовой результат и оценку погрешности.

Консольный UnitTest завершился с 135 успешными проверками и без ошибок. Все 30 сравнений с mpmath прошли цели 4, 8 и 10 знаков после запятой.

Сборка и запуск

Из корня проекта `EcoIntegralSimpson`:

```
cd AssemblyFiles/Linux/gcc_v132
./build.sh test
./build.sh verify
```

`test` собирает компонент и запускает C-клиент. `verify` дополнительно выполняет внешний компаратор, сравнение с mpmath и аудит объектов. Оболочка использует установленные в `~/Dev/toolchain` GCC и make, а также Framework в `~/Dev/Eco.Framework`. Предварительная активация `activate_eco.sh` для этих команд не нужна.

9. Использование ИИ в работе

При выполнении лабораторной использовался ИИ-агент OpenAI Codex: модель GPT-6.1 Sol (`gpt-6.1-sol`), уровень reasoning — `xhigh`. Работа с агентом строилась итеративно. Сначала выполнялись поиск и анализ математических материалов по возможным численным методам, обсуждались обоснование формул, оценки погрешности и условия их применения. Затем согласовывались математическая модель, критерий остановки и интерфейс компонента. На основе этих решений генерировались прототипы кода компонента и его тестов. Работоспособность проверялась сборкой, запуском UnitTest и сравнением с mpmath; результаты проверок использовались для уточнения реализации.

Для явного хранения контекста использовались отдельные Markdown-файлы: `TASK.md` с требованиями, `docs/DESIGN.md` с математическими и архитектурными решениями, `AGENTS.md` с инструкциями для агента. Эти документы позволяли продолжать работу с учётом уже согласованных решений и ограничений, сохраняя контекст между итерациями.